

CS F407 – Search Lab: Warehouse Robot Navigation with A*

Name: Aditeya Kayal | **ID:** 2024A8PS0689G | **Course:** CS F407 Artificial Intelligence

This report presents the solution to the laboratory exercise [search_lab.pdf](#).

File	Contents
warehouse_search.py	Problem representation, A*, BFS, heuristics, path validator, and all experiments
results.txt	Full output of <code>python warehouse_search.py</code> ; every number in this report is taken from it
README.md / REPORT.pdf	This report, in Markdown and PDF form

Requirements: Python 3 (standard library only). To reproduce every result:

```
python warehouse_search.py
```

Task 0 – Formulation of the search problem

Component	Specification
State S	The robot's grid position (row, column), restricted to free cells (. , S , G). The warehouse has 64 free cells.
Actions A	Up, Down, Left, Right
Transition T	$T((r, c), a)$ = the adjacent cell in direction a, provided that cell lies inside the grid and is not an obstacle #
Initial state S_0	The cell marked S: (1, 1)
Goal G	The cell marked G: (7, 15)
Cost c	1 per move; path cost equals the number of moves

- (a) Information needed to specify a state.** Only the robot's position (row, column). The map is static, so it belongs to the problem definition rather than to the state.
- (b) What makes an action invalid.** An action is invalid when the target cell is an obstacle or lies outside the grid.
- (c) Is the problem deterministic?** Yes. Each action from a given state has exactly one outcome. The environment is also fully observable, static, and discrete.

(d) What constitutes a solution. A sequence of actions that leads from s_0 to G , passing only through free cells. An *optimal* solution is one with the minimum number of moves.

Task 1 – Design of the agent

1. **State representation.** A tuple `(row, col)`. Tuples are hashable, so they can be used directly in sets and dictionaries.
2. **Warehouse representation.** A list of character lists, parsed from the ASCII map by the `Warehouse` class, which also locates `S` and `G`.
3. **Valid actions.** `Warehouse.successors(state)` applies each of the four displacements and keeps only the results for which `is_free()` is true.
4. **Goal recognition.** `Warehouse.is_goal(state)` compares the state with the goal cell. The test is applied when a state is *removed* from the frontier, not when it is generated. For A* this ordering is required to guarantee optimality.
5. **Frontier contents.** Priority-queue entries `(f, counter, state)`. The `counter` breaks ties among equal f -values in first-in, first-out order. The values $g(n)$ and the parent pointers are stored in the dictionaries `g` and `parent`.
6. **Path reconstruction.** `reconstruct(parent, goal)` follows the parent pointers from the goal back to the start and reverses the list.

Reported on termination: whether a solution was found, the path (as cells and as actions), its length, the number of states expanded, and an independent validity check (`validate_path`).

Task 2 – LLM-generated A* implementation

LLM used: Claude (Anthropic).

Prompt (based on the design above):

I am implementing a simple goal-based search agent in Python. The environment is a grid represented by an ASCII map. The agent starts at S and must reach G. The symbols # represent obstacles and . represents free cells. The agent can move up, down, left, or right, and every movement has cost 1. Implement A* search. Use Manhattan distance as the heuristic, $h(n) = |x - x_G| + |y - y_G|$. The program should represent grid positions as states; maintain an appropriate frontier; calculate $g(n)$, $h(n)$ and $f(n)$; avoid repeatedly expanding the same state; reconstruct the path when the goal is reached; report the path and its length; and report the number of states expanded. Keep the implementation simple and explain the main components of the code.

Task 3 – Testing the generated program

Every returned path was also checked by `validate_path()`. This function is independent of the search code. It confirms that the path starts at S, ends at G, uses only free cells, and moves exactly one cell per step. Optimality was checked by comparing the path length against BFS, which is optimal for unit costs.

Test	Map	Solution found	Path length	States expanded	Valid	Optimal
1. Original warehouse	17 × 9 lab map	Yes	40	64	Yes	Yes (BFS: 40)
2. Trivial case	#SG##	Yes	1	2	Yes	Yes
3. No solution	G enclosed by walls	No, reported as failure	–	9	–	–
4. Alternative paths	4-move upper route and 8-move lower route	Yes	4	5	Yes	Yes (BFS: 4)

Test 1 path (* marks the route):

```
#####
#S*****#
#.###*#####*#
#...#####*#
###.*#####*#
#...#####*#
#.#####.*#
#.....#G#
#####
```

Actions: Right ×4, Down ×4, Right ×8, Up ×2, Left ×6, Up ×2, Right ×8, Down ×6 (40 moves).

In Test 3 the program expanded all 9 reachable cells, emptied the frontier, and terminated with a failure report rather than looping. In Test 4 it chose the 4-move upper route over the 8-move lower route.

Task 4 – Inspection of the A* algorithm

Concept	Location in <code>warehouse_search.py</code>
State	Tuple <code>(row, col)</code> ; the start and goal are found in <code>Warehouse._find()</code>
Action	The <code>ACTIONS</code> dictionary (name $\rightarrow$ displacement)
Transition	<code>Warehouse.successors()</code> together with <code>Warehouse.is_free()</code>
Goal test	<code>Warehouse.is_goal()</code> , called in <code>astar()</code> immediately after a state is popped
$g(n)$	Dictionary <code>g</code> ; updated as <code>new_g = g[state] + 1</code>
$h(n)$	Functions <code>h_manhattan</code> (default), <code>h_zero</code> , <code>h_euclidean</code> , <code>h_double_manhattan</code> , passed as the parameter <code>h</code>
$f(n)$	<code>f_next = new_g + h(nxt, goal)</code>
Frontier	Binary heap <code>frontier</code> (<code>heapq</code>) of <code>(f, counter, state)</code>
Visited states	Set <code>expanded</code> (closed set)
Path reconstruction	<code>reconstruct(parent, state)</code>

(a) The frontier is a binary min-heap (priority queue) implemented with `heapq`. **(b)** `heapq.heappop` returns the entry with the smallest $f(n)$; ties are broken by insertion order through `counter`. **(c)** The heuristic is evaluated when a successor is generated, in the line `f_next = new_g + h(nxt, goal)`. **(d)** Yes; $f(n) = g(n) + h(n)$ is computed explicitly and used as the priority. **(e)** Repeated exploration is prevented in two ways. A successor is pushed only if the new path to it is cheaper than any known path (`new_g < g[nxt]`). A popped state that is already in the closed set `expanded` is discarded as a stale entry.

Task 5 – Comparison of A* with blind search (BFS)

Measure	BFS	A* (Manhattan)
Solution found	Yes	Yes
Path length	40	40
States expanded	64	64

(a) Both algorithms found a solution. **(b)** Both paths have length 40, which is optimal. **(c)** On this map both algorithms expanded all 64 free cells, so neither expanded fewer. **(d)** *Why A* would usually expand fewer states, and why it did not here.* A* expands states in order of $f(n) = g(n) + h(n)$, so it concentrates on states that appear to lie on a short route to the goal. With a consistent heuristic it must expand every state with $f(n) < C^*$, where C^* is the optimal cost (40). On this warehouse the Manhattan distance from S to G is only 20, but the true shortest path is 40, because the only entrance to G is from the top of the right-hand corridor. The heuristic therefore underestimates the true cost substantially. A direct computation shows that **every one of the 64 free cells has $f(n) \leq 40$** (49 cells have $f < 40$, and 15 have $f = 40$). A* was therefore obliged to expand the entire map before reaching the goal.

A* does save work when the heuristic is more informative, as the second map in Task 6 shows: 18 expansions with Manhattan against 28 with $h = 0$ (equivalent to uniform-cost search).

Task 6 – Investigation of the heuristic

Why Manhattan distance is appropriate. With only horizontal and vertical unit moves, any path from n to G must make at least $|x - x_G|$ horizontal moves and $|y - y_G|$ vertical moves. The Manhattan distance is therefore never greater than the true cost, so it is **admissible**. A single move changes it by exactly 1, so it is also **consistent**. It is the exact cost when no obstacles are present.

The four heuristics were evaluated on the original warehouse and on a second, more open map (included in the program as `SECOND_MAP`).

Original warehouse (optimal length 40):

Heuristic	Solution found	Path length	States expanded	Optimal
$h = 0$	Yes	40	64	Yes
Manhattan	Yes	40	64	Yes
Euclidean	Yes	40	64	Yes
$2 \times \text{Manhattan}$	Yes	40	64	Yes

Second map (optimal length 11):

Heuristic	Solution found	Path length	States expanded	Optimal
$h = 0$	Yes	11	28	Yes
Manhattan	Yes	11	18	Yes
Euclidean	Yes	11	19	Yes
$2 \times \text{Manhattan}$	Yes	13	15	No

$2 \times \text{Manhattan}$ (length 13) <pre>##### #S#...#.# #*#.#...# ##*...***# #.*****# ##.####.G# #####</pre>	Manhattan (length 11) <pre>##### #S#...#.# #*#.#...# ##*##### #.....*# ##.####.G# #####</pre>
---	--

Observations.

1. **$h = 0$** turns A* into uniform-cost search. It remains optimal but expands the most states (28 on the second map), because it uses no information about the goal's location.
2. **Euclidean distance** is admissible, since the straight-line distance never exceeds the grid distance, but it is less informed than Manhattan distance on a 4-connected grid. It gave the optimal path with

one more expansion (19 against 18).

3. **2 × Manhattan** overestimates the true cost and is therefore **not admissible**. It expanded the fewest states (15), but on the second map it returned a path of length 13 instead of 11. Doubling h gives the heuristic more weight than the accumulated cost g , so the search behaves more greedily: it commits early to states that appear close to the goal and does not return to explore the cheaper alternative.

Conclusion. An admissible heuristic guarantees an optimal solution, and among admissible heuristics a more informed one, closer to h^* , expands fewer states. A heuristic that is too aggressive ($h > h^*$) can reduce the search effort but forfeits the optimality guarantee. On the original warehouse this effect did not appear, because the maze structure forces every heuristic to explore the whole map.

Task 7 – Evaluation of the LLM-generated agent

1. **Correct immediately.** The problem representation, the successor function, the A* loop with a closed set, and path reconstruction were all correct on the first execution. All four tests passed without modification.
2. **Bugs or design problems.** No functional errors were found. Two design decisions were verified explicitly rather than accepted on trust. First, the goal test must be applied when a state is popped, not when it is generated; otherwise A* can return a suboptimal path. Second, duplicate heap entries must be discarded when popped, because `heapq` has no decrease-key operation.
3. **How problems would be discovered.** Through tests with known answers (the trivial and unreachable maps), comparison against BFS as an independent optimality reference, and the separate `validate_path` check.
4. **Unfamiliar terminology.** The terms *stale entry* and *closed set* required clarification; both refer to avoiding redundant expansions.
5. **Modifications.** The code was extended to accept the heuristic as a parameter (for Task 6), to add a BFS variant using the same problem interface (for Task 5), and to add an independent path validator. The first map chosen to demonstrate an inadmissible heuristic, an open room, produced an optimal path with every heuristic and so demonstrated nothing. It was replaced by a map found through a randomised search over small grids, on which 2 × Manhattan is measurably suboptimal.
6. **Most useful tests.** The comparison with BFS, which provides an independent optimality check, and the unreachable-goal test, which verifies termination.
7. **Could the program be trusted without testing?** No. A path that merely looks reasonable can still be suboptimal, as the 2 × Manhattan result shows: it is valid but two moves longer than necessary. Such an error cannot be seen without a reference solution.
8. **What was learned about A*.** A*'s advantage over blind search depends entirely on how well $h(n)$ reflects the true remaining cost. On a maze that forces a long detour, even an admissible and consistent heuristic provides no saving, and A* expands exactly as many states as BFS.

Summary of responsibilities

Category	Items
Designed by the student	Problem formulation (Task 0), agent design (Task 1), test cases, choice of validation criteria
Suggested by the LLM	Implementation code for A*, the heap-based frontier, stale-entry handling
Accepted	Core A* and BFS implementations
Changed	Heuristic parameterisation, BFS variant, independent path validator, replacement of the second test map
Tested	Four specified tests, BFS comparison, four heuristics on two maps

Final reflection

1. Why formulate the problem before writing the algorithm? The formulation fixes what the algorithm must operate on: what a state is, which actions are legal, what counts as reaching the goal, and how cost is measured. Without it, one cannot judge whether generated code is correct, because there is no specification to compare against. In this laboratory, for example, the decision to model the state as a position alone kept the state space to 64 cells. The unit step cost is what made BFS a valid optimality reference.

2. In what sense is A* an "informed" search algorithm? A* uses knowledge about the goal, the heuristic $h(n)$, in addition to the cost already incurred, $g(n)$. Blind algorithms such as BFS and DFS order the frontier only by depth or insertion order. A* orders it by an estimate of total solution cost through each node, so it prefers states that appear to lie on short routes to the goal.

3. Why does the choice of heuristic matter? The heuristic determines both efficiency and correctness. On the second map, $h = 0$ expanded 28 states, Manhattan 18, and $2 \times \text{Manhattan}$ 15. Only the admissible heuristics guaranteed the optimal length of 11; the inflated heuristic returned 13. A good heuristic is therefore admissible, so that optimality is preserved, and as informed as possible, so that search effort is reduced.

4. What did the LLM contribute to the engineering process? It translated the design into working Python quickly: the heap-based frontier, the bookkeeping for g -values and parent pointers, and path reconstruction. It also explained why Manhattan distance is admissible for 4-connected movement. This allowed the laboratory time to be spent on designing tests and analysing behaviour rather than on writing boilerplate code.

5. What could go wrong if LLM-generated code were accepted without testing? The code could return valid-looking but suboptimal paths (for example, through goal testing at generation time or an inadmissible heuristic), loop indefinitely on unreachable goals, or report misleading search statistics. None of these faults is visible from a single plausible output. They are revealed only by tests with known answers and by independent references such as BFS and the path validator.